Python Portfolio: Erarbeitung/Reflexion

IU Internationale Hochschule

1 Untersuchung der Umsetzung objektorientierter Konzepte

In diesem Abschnitt wird mit kleinen Testprogrammen geprüft, wie die in Phase 1 beschriebene Architektur mit Python umgesetzt werden kann. Die Imports von Bibliotheken werden hier in den Testprogrammen weggelassen, um die Testprogramme übersichtlich zu halten.

1.1 Umsetzung der Klassen mit @dataclass

Im folgenden Testprogramm wird geprüft, wie die Klasse Prüfungsleistung als Dataclass implementiert werden kann:

```
@dataclass
class Prüfungsleistung:
  datum: date
  note: float
```

Listing 1: Implementierung der Klasse Prüfungsleistung

Der Dekorator **@dataclass** vereinfacht die Deklaration der Klassen und spart somit Boilerplate-Code. Ohne diesen müsste die Klasse wie folgt implementiert werden:

```
class Prüfungsleistung:
  def __init__(self, datum: date, note: float):
    self.datum = datum
    self.note = note
```

Listing 2: Klasse Prüfungsleistung ohne @dataclass Dekorator

Zudem wird dadurch die JSON-Serialisierung mit verschachtelten Klassen vereinfacht.

1.2 Umsetzung der Beziehungen (Aggregation und Komposition)

In den folgenden Testprogrammen werden nicht alle Eigenschaften der Klassen angegeben.

Die Klasse Studiengang hat über eine Komposition mehrere Semester. Für die Modellierung dieser Beziehung eignet sich in Python eine Liste:

```python
@dataclass
class Semester:
  nummer: int

@dataclass
class Studiengang:
  semester: list[Semester] = field(default_factory=lambda: [])
```

Listing 3: Komposition Studiengang - Semester

Mit **list[Semester]** kann per Typing angegeben werden, dass der Datentyp eine Liste aus Semesterklassen sein soll. Zusätzlich wird über **field(default_factory=lambda: [])** eine leere Liste als Standardwert definiert.

Die Aggregation zwischen Semester und Modul wird im gleichen Stil umgesetzt. Hierbei ist es in Python nicht möglich, im Code darzustellen, dass es sich um eine Aggregation handelt. Dieser Aspekt bleibt rein konzeptuell.

Die Komposition zwischen Modul und Prüfungsleistung, bei der entweder keine oder maximal eine Prüfungsleistung zugelassen ist, wird durch eine optionale Eigenschaft **prüfungsleistung** in der Modulklasse umgesetzt:

```python
@dataclass
class Prüfungsleistung:
  note: float

@dataclass
class Modul:
  prüfungsleistung: Prüfungsleistung | None = None
```

Listing 4: Komposition Modul - Prüfungsleistung

Da nur maximal eine Prüfungsleistung zugelassen wird, kann auf eine Liste verzichtet werden. Um auszudrücken, dass dieses Objekt auch fehlen kann, wird der Typ von Prüfungsleistung mit **| None** gekennzeichnet.

2 Gesamtarchitektur des Prototypen

In diesem Abschnitt werden die einzelnen Teilbereiche der Anwendung als UML-Diagramm dargestellt und untersucht.

Es wurde entschieden, nur die Entitäten und deren Eigenschaften in Deutsch zu benennen. Somit gibt es eine klare Regel, wann welche Sprache verwendet wird.

2.1 Datenmodellierung mit Entitäten

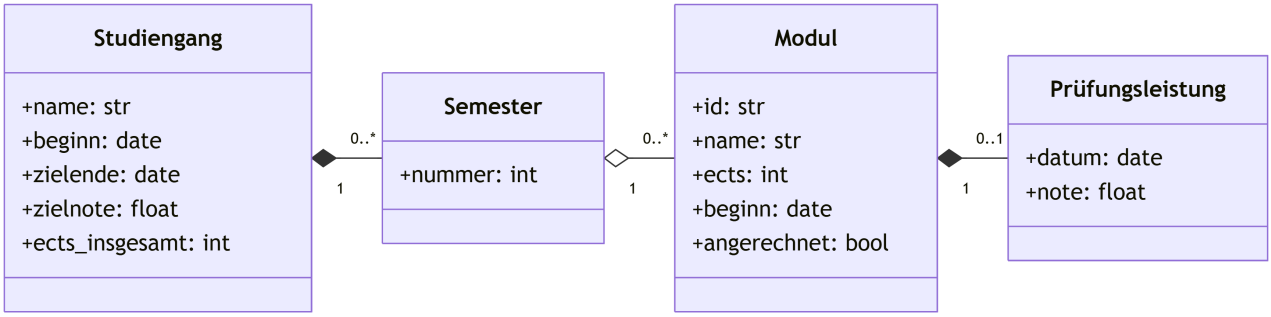

Abbildung 1: Entitäten

Weitestgehend wird das UML-Diagramm aus Phase 1 übernommen. Lediglich die Eigenschaft *ende* der Klasse Modul wurde entfernt, da eine Redundanz mit der Eigenschaft *datum* aus Prüfungsleistung erkannt wurde.

Die Klassen dienen lediglich der Datenhaltung und besitzen daher keine Methoden.

2.2 Klassen für Geschäftslogik und Persistenz

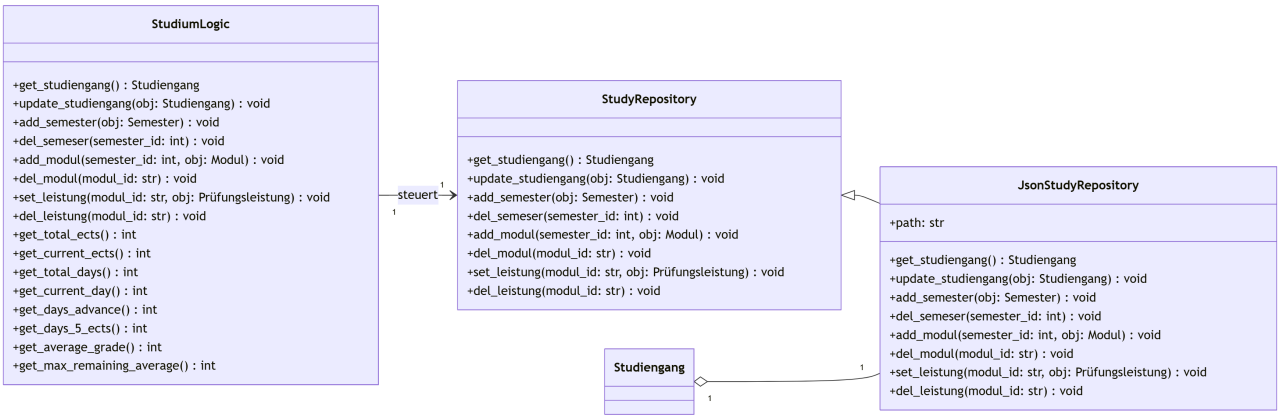

Abbildung 2: Klassen für Geschäftslogik und Persistenz

StudyLogic bildet den Kern der Anwendung. In dieser Klasse befindet sich die gesamte Geschäftslogik der Anwendung mit sämtlichen Berechnungsmethoden, die für die Anzeige notwendig sind. Die Berechnungsmethoden für die Anzeige greifen auf die Methoden der *StudyRepository* zu, um Daten zur Berechnung zu erhalten. Die CRUD (Create, Read, Update und Delete) Methoden leiten weitestgehend ohne Logik die Aufrufe zur *StudyRepository* weiter.

StudyRepository ist eine Abstraktion und beinhaltet Basismethoden, die der Datenhaltung dienen. Diese Klasse dient nur als Schnittstelle, um vorzugeben, welche Methoden eine erbende Klasse implementieren muss. **JsonStudyRepository** implementiert diese Methoden für die Speicherung der Daten in einer JSON-Datei. Durch dieses Design, können künftig einfach Klassen zur Datenhaltung in anderen Systemen implementiert werden. Beispielsweise könnte eine *OracleDbStudyRepository* implementiert werden.

Ziel dieses Designs ist die strikte Trennung der verschiedenen Domänen Anzeige-, Geschäfts- und Persistenzlogik (Separation of concerns). Somit werden zukünftige Erweiterungen vereinfacht und die Integration von beispielsweise Unit-Tests ermöglicht.

2.3 Klassen für die Anzeige

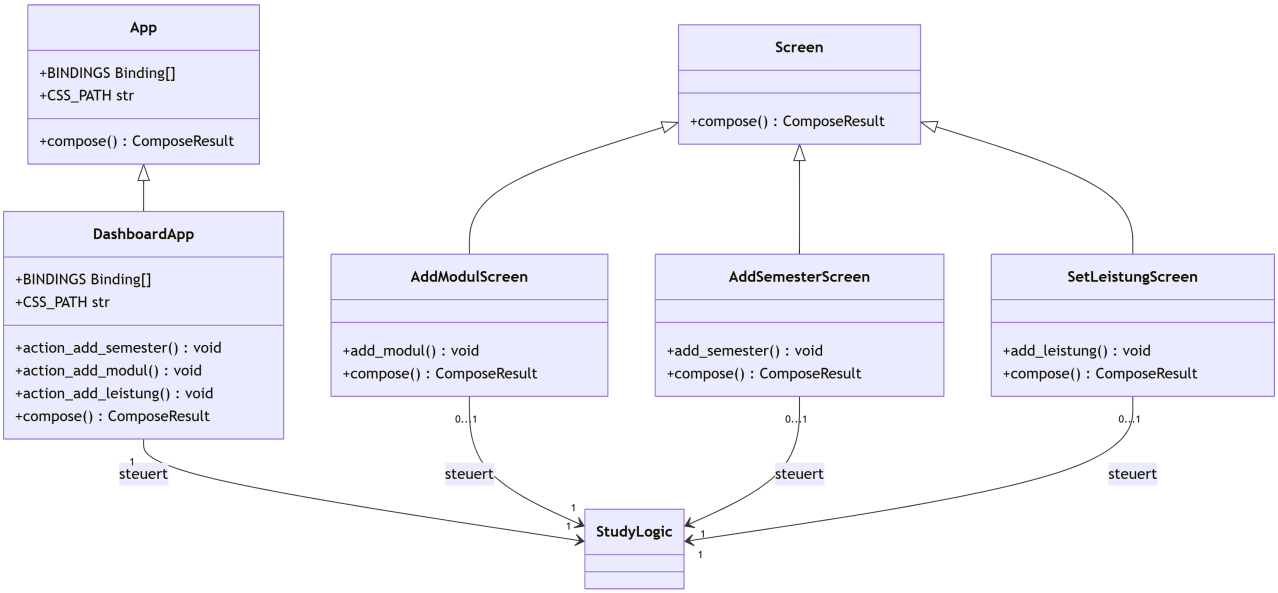

Abbildung 3: Klassen für die Anzeige

DashboardApp ist die Hauptklasse für die Anzeige und erbt von der Basisklasse *App*, die von der Bibliothek textual zur Verfügung gestellt wird. Sie implementiert das Hauptfenster des Dashboards und beinhaltet alle zugehörigen Anzeigeelemente. Diese Anzeigeelemente werden durch die Klasse *compose* konfiguriert. Mit der Eigenschaft *BINDINGS* werden die Tastaturevents aus der Fußzeile definiert. Bei entsprechender Betätigung, werden die Methoden *action_add_semester*, *action_add_modul* und *action_add_leistung* aufgerufen. *CSS_PATH* definiert eine Datei, die im CSS-Stil das Design der UI beschreibt. Die beschriebenen Eigenschaften und Methoden werden automatisch von der Bibliothek textual ausgewertet und ausgeführt.

AddSemesterScreen, **AddModulScreen** und **SetLeistungScreen** sind weitere Seiten, die von DashboardApp konstruiert und angezeigt werden, sobald der Benutzer entsprechende Aktionen fordert. Zusätzliche Seiten, die erst im weiteren Verlauf der Anwendung angezeigt werden, müssen von der Basisklasse *Screen* von textual erben. Sobald der Benutzer alle benötigten Daten eingegeben hat, wird die Anpassung über die Klasse *StudyLogic* persistiert.

Die Basisklassen **App** und **Screen** stellen viele weitere Eigenschaften und Methoden zur Verfügung, die überschrieben werden können. Jedoch sind keine weiteren Eigenschaften oder Methoden für das Programm relevant und werden daher nicht im UML-Diagramm aufgeführt.